LECTURE NOTES · PRETRAINING

Scaling Laws, Visually

Definitions, intuition and worked examples

Pretraining: the initial stage of learning to predict text from many examples.

Scaling law: a measured relationship between resources and prediction error.



Loss = prediction error. Compute = arithmetic work. Log_{10} = a scale where each step means 10×. Green = small-run ladder. Orange = forecast. Illustration only.

The resources and the outcome

A scaling law asks how prediction error changes when we spend more resources.

N: model size

Parameter: a learned numerical weight inside the model. N counts these weights.

Capacity: the patterns the model can represent. More weights usually allow richer patterns.

Dense transformer: a text model that uses essentially all its main weights for each token.

D: training data

Token: a unit of text, such as a word piece, punctuation mark or character.

D: the total tokens processed during training, including repeated tokens.

Tokenizer: the rule that splits text into tokens. Its choices change the unit of loss.

C: training compute

FLOP: one floating-point arithmetic operation. C counts operations over the run.

C ≈ 6ND: a useful estimate for dense transformer training.

Loss L: average prediction error on held-out text, which training has not used. Lower is better.

Intuition. A larger model is a larger notebook. Data supplies examples to learn from. Compute pays for processing those examples. Loss tells us how well the model learned.

Kaplan et al. (2020); Hoffmann et al. (2022)

Loss measures surprise at the correct token

The model assigns probabilities to possible next tokens. Confident correct predictions receive a smaller penalty.

$$\text{Token loss} = -\ln(p)$$

Example: p = 0.5: The penalty is 0.693 nats. A correct token the model expected only half the time is moderately surprising.

Example: p = 0.1: The penalty is 2.303 nats. Low confidence in the correct token costs more.

Average loss L: Average these penalties over the same held-out text for every model. A small change can reflect many improved predictions.

Definitions on this slide

p: probability the model assigns to the actual next token.

ln: natural logarithm, using base e ≈ 2.718.

Nat: the information unit produced by a natural logarithm.

NLL / cross-entropy: negative log-likelihood. For next-token evaluation, average NLL is the usual loss.

Bits per byte: total NLL divided by ln(2) and by the number of text bytes. Useful across tokenizers.

Intuition. Loss gives partial credit. A model can become less surprised by the right answer long before its single most likely answer changes.

A power law gives the same gain for the same ratio

On logarithmic axes, a pure power law becomes a straight line.

$$L = A\,x^{-\alpha}$$



One step on a $\log_{10}$ axis means 10× as much. The line slopes downward at $-\alpha$.

Definitions on this slide

x: the resource being varied, such as parameter count, tokens or compute.

A > 0: the fitted scale of the curve, in the chosen units.

α > 0: the exponent controlling how quickly loss falls.

Logarithm / $\log_{10}$: the exponent of 10: $\log_{10}(1{,}000) = 3$.

Decade / ∝: a decade is a factor of 10. ∝ means "proportional to."

Intuition. If $\alpha = 0.05$, multiplying the resource by 10 multiplies loss by $10^{-0.05} \approx 0.891$. That is an 11% reduction each time, rather than a fixed number of loss units.

The floor is the part that further scaling cannot remove

A straight line applies to excess loss after subtracting the floor.

$$L = E + A\,x^{-\alpha}$$



The total-loss curve bends as it approaches E. The excess-loss curve remains straight.

Definitions on this slide

L: average prediction loss on a fixed evaluation set.

E: the fitted limiting loss, also called the floor or asymptote.

L – E: excess loss: the part this fitted law says scaling can reduce.

A, x, α: curve scale, resource amount and positive decay exponent.

Regime change: a range where a different relationship starts to apply.

Intuition. A blurry camera can improve until it reaches the scene's unavoidable ambiguity. Subtracting that ambiguity reveals the part better equipment can still fix.

Learning curves before language models

In the 1990s, researchers already used small datasets to estimate behavior at larger scale.

$$\text{error}(n) \approx a \pm b\, n^{-\alpha}$$



Cortes et al. fit training and test curves to estimate their shared limiting error.

Definitions on this slide

n: the number of training examples.

Training / test error: mistakes on examples used for learning / fresh examples.

a: the estimated long-run error. b sets the gap. α sets its decay rate.

Generalization: performance on new examples from the same population.

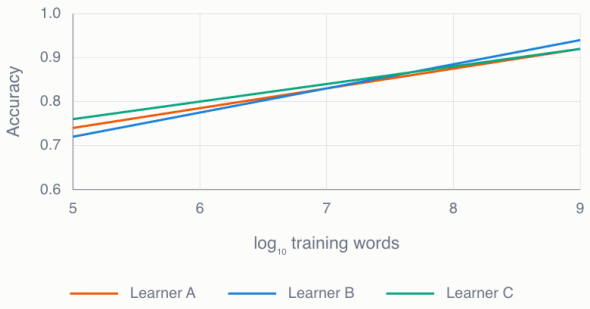
Asymptotic rate: how error approaches its limit as n becomes very large.

Intuition. A tiny dataset is easy to memorize, so training error can look excellent while test error is poor. More examples narrow that gap.

Cortes et al. (1993); Amari (1992–93); Vapnik learning theory; illustrative curve shapes

More data can change which model wins

Early language and vision studies showed that algorithm rankings can change with dataset size.



Banko and Brill (2001) studied word-choice disambiguation from about 1 million to 1 billion words.

Definitions on this slide

Accuracy: the fraction of predictions counted as correct.

Learner: a particular learning algorithm or model.

Corpus: a collection of text used for training or evaluation.

NLP: natural language processing: computational tasks involving human language.

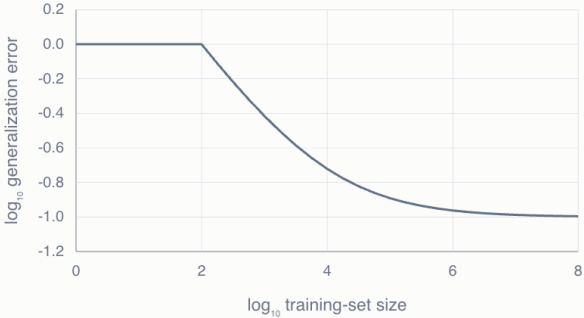
Log-linear: a straight relationship between an outcome and the logarithm of a resource.

Intuition. A model that learns slowly from a little data may eventually overtake one that learns quickly but runs out of useful capacity.

Banko & Brill (2001); Halevy et al. (2009); Sun et al. (2017); illustrative curve shapes

Deep learning curves have distinct regions

Hestness et al. found broadly similar shapes across language, speech and vision tasks.



Little data: weak learning. Middle range: regular improvement. Large data: approach to a floor.

Definitions on this slide

Generalization error: error on unseen examples. The exact metric depends on the task.

Power-law region: the range where resource ratios predict regular error reductions.

Irreducible error: the limiting error within the assumptions of the task and model.

Architecture: the arrangement of layers and computations in a model.

Extrapolation: prediction outside the range of measured examples.

Intuition. A road can be straight for a long stretch and still bend later. Estimate the slope in the region relevant to the planned run.

Hestness et al. (2017); Rosenfeld et al. (2019); illustrative curve shapes

Kaplan's three empirical laws

L is held-out prediction loss. Each fit describes one limiting regime in the 2020 experiments.

Model size

$L(N) = (N_c / N)^{\alpha_N}$: N counts non-embedding parameters. Embeddings map tokens to internal vectors.

$\alpha_N = 0.076$: With ample data and training, 10× the parameters predicts about 16% lower loss.

$N_c = 8.8 \times 10^{13}$: a fitted normalization constant, not a recommended model size.

Dataset size

$L(D) = (D_c / D)^{\alpha_D}$: D counts training tokens. This fit considers sufficiently large, trained models.

$\alpha_D = 0.095$: 10× the data predicts about 20% lower loss in this regime.

$D_c = 5.4 \times 10^{13}$: the fitted data normalization.

Training compute

$L(C) = (C_c / C)^{\alpha_C}$: C is compute along the study's efficient training frontier. Cc normalizes the curve.

$\alpha_C = 0.050$: 10× the compute predicts about 11% lower loss.

PF-day: 10^{15} operations/second for one day, or 8.64×10^{19} FLOPs.

Intuition. A bottleneck is the resource holding progress back. Adding books helps only if the model and training process can use them.

Kaplan et al. (2020), fitted exponents and single-resource normalizations

The joint law connects model and data bottlenecks

A joint fit allows either limited model size or limited data to control the loss.

$$L(N,D) = [(N_c/N)^{(\alpha N/\alpha D)} + D_c/D]^{\alpha D}$$

Model-limited: When data is plentiful, the data term becomes small and the model-size term controls loss.

Data-limited: When the model is large, the finite-data term becomes relatively more important.

Overfitting: A model improves on its training examples without a matching improvement on new examples. Kaplan linked its onset to a growing $N^{0.74}$ / D.

Definitions on this slide

L(N,D): loss as a function of parameter count N and training-token count D.

Nc, Dc: normalizations fitted for this joint equation. Do not reuse other fits blindly.

αN, αD: positive exponents for the model and data dependence.

^ and brackets: ^ means “raised to a power.” Brackets group the terms before exponentiation.

Depth / width: number of layers / size of a layer’s internal representation.

Intuition. Two shortages can coexist. A huge model with too few examples can be as wasteful as abundant data paired with a model too small to learn it.

Kaplan et al. (2020)

Two ways to spend 10× more compute

Kaplan favored growing model size faster. Chinchilla later favored roughly balanced growth.

Allocation rule	Model size N	Tokens D
Kaplan (2020)	×5.37	×1.86
Chinchilla (2022)	×3.16	×3.16

Definitions on this slide

C: training compute, approximately 6ND for a dense model.

$N \propto C^{0.73}$: Kaplan's model-size allocation rule. $D \propto C^{0.27}$ supplies the remaining exponent.

$N, D \propto C^{0.5}$: the approximate balanced Chinchilla rule.

Convergence: training long enough that further updates yield little improvement.

GPT-3: 175B parameters and 300B tokens, about 1.7 tokens/parameter. B means billion (10^9).

Intuition. The multipliers multiply to 10 because compute is proportional to N × D. The disagreement concerns how to split the same budget.

Kaplan et al. (2020); Hoffmann et al. (2022)

Batch size trades update count against data per update

D counts processed tokens and C counts training operations. Larger batches reduce noise, with diminishing returns.

$$D = B \times S$$

Same tokens, fewer updates: If batch B doubles and total tokens D stay fixed, the number of updates S halves. This identity alone does not guarantee the same final loss.

Critical batch size: Below this approximate threshold, larger batches can reduce update count efficiently. Above it, extra examples per update buy less progress.

Kaplan's fitted allocation: $B \propto C^{0.24}$ and $S \propto C^{0.03}$, so $D \propto C^{0.27}$. These are historical fitted relationships.

Definitions on this slide

B: batch size, measured here in tokens processed per update.

S: number of optimizer updates, also called training steps.

Optimizer: the algorithm that updates the learned weights.

Gradient: a signal indicating how changing weights would change loss.

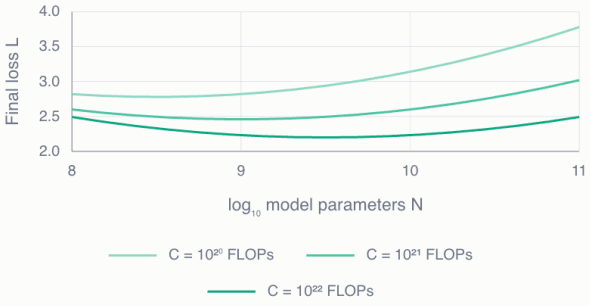
Noise: random variation in that signal from sampling a finite batch.

Intuition. Ask more people before making a decision and the estimate becomes steadier. Once it is steady enough, asking twice as many people may mostly add cost.

Kaplan et al. (2020); McCandlish et al. (2018)

Finding the best split at a fixed compute budget

Hoffmann et al. trained more than 400 models and compared three ways to locate the optimum.



On each curve, C is fixed. A larger N leaves fewer training tokens: D = C / (6N).

Definitions on this slide

Compute-optimal: lowest prediction loss at a fixed training-compute budget.

N / D / C: model parameters / processed tokens / total operations.

IsoFLOP profile: a curve comparing runs with the same operation count.

Lower envelope: the best loss reached at each compute budget.

Parametric fit: an equation with fitted constants. The valley minimum is the preferred model size.

Intuition. A tiny model cannot use all the available information. An enormous model leaves too little budget to teach it. The valley balances these shortages.

Hoffmann et al. (2022); curves are illustrative

Chinchilla trained a smaller model on more tokens

At approximately the same reported training cost, the 70B model outperformed the 280B model on most evaluations.

Model	Parameters N	Tokens D	D / N
Gopher	280B	300B	1.1
Chinchilla	70B	1.4T	20

Definitions on this slide

B / T: billion (10^9) / trillion (10^{12}).

D / N: training tokens processed per model parameter.

C: reported compute: about 5.8×10^{23} FLOPs for each model. Detailed counts differ from 6ND.

Serving: running the trained model to produce predictions for users.

Approximate rule: $N \propto C^{0.5}$ and $D \propto C^{0.5}$. With $D \approx 20N$, $N \approx \sqrt{(C / 120)}$.

Intuition. More capacity is useful only when training supplies enough examples. Better teaching can let a smaller model beat a much larger one.

Hoffmann et al. (2022), Gopher–Chinchilla comparison

Why dense training costs roughly 6ND

The estimate counts the arithmetic needed to make predictions and update the weights.

$$C \approx (2N + 4N) \times D = 6ND$$

Forward pass: about 2N FLOPs/token: Use weights to calculate a prediction. Counting a multiply and an addition separately gives roughly two operations per weight.

Backward pass: about 4N FLOPs/token: Propagate the prediction error backward and compute how weights should change. This often costs about twice the forward pass.

Scope of the estimate: Long sequences, vocabulary layers, sparse routing and other operations can change the count. Use measured or detailed compute for final decisions.

Definitions on this slide

C, N, D: training operations, model parameters and processed tokens.

FLOP: one floating-point operation, such as a multiplication or addition.

Activation: an intermediate value inside the model.

Backpropagation: calculating how each weight contributes to prediction error.

Attention / context: relating tokens to one another / the sequence being considered.

Intuition. Training reads each weight to make a guess, then does extra work to learn from the error. Doing this for every token explains the N × D product.

Dense-transformer approximation; detailed compute conventions vary by study

Methodology explains much of the allocation disagreement

Porian et al. reproduced the gap and isolated key experimental choices.

Count the work

Embedding / output layers: weights that map tokens into vectors or vectors back to token scores.

Small-model effect: these layers can be a large fraction of a small model's compute.

Consequence: excluding their cost can distort the inferred advantage of larger models.

Tune each scale

Learning rate: the size of each weight-update step.

Warmup: an initial period that gradually raises the learning rate.

Optimizer tuning: choosing update settings, including how gradients are smoothed, separately at each scale.

Check the fit

Schedule horizon: the intended training duration over which the learning rate changes.

Replication: repeating an analysis to test whether its result holds.

Published fit issue: Besiroglu et al. found that one Chinchilla fit implied about 70 tokens/parameter at relevant scale. Their refit aligned near 20.

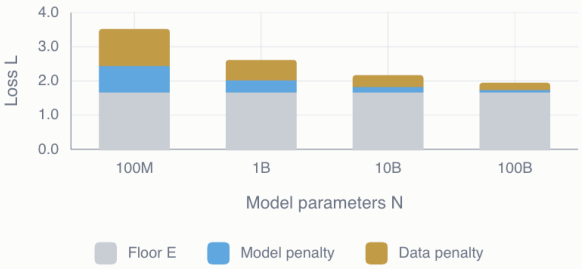
Intuition. A poorly tuned small run is an unfair competitor. A curve fitted through unfair comparisons can make a larger model look more efficient than it is.

Porian et al. (2024); Besiroglu et al. (2024)

The parametric loss model, term by term

Each term describes a different limitation. The interpretation is useful even when the fitted constants are uncertain.

$$L(N,D) = E + A / N^{\alpha} + B / D^{\beta}$$



Published fit at D = 20N. The floor takes a larger share as the two penalties shrink. M/B = million/billion.

Definitions on this slide

L, N, D: held-out loss, model parameters and training tokens.

E: fitted limiting loss. It can include intrinsic uncertainty and model mismatch.

A / N^{α}: finite-model penalty. A sets its scale and α its decay rate.

B / D^{β}: finite-data penalty. B sets its scale and β its decay rate.

Fit caveat: the published coefficients differ from a later replication. The decomposition is an empirical model.

Intuition. Imagine two leaks feeding the excess loss. Making the model bigger closes one leak. Adding data closes the other. The floor remains underneath.

Hoffmann et al. (2022), with the replication caveat of Besiroglu et al. (2024)

Balancing the two penalties gives the allocation rule

With training compute fixed, improving one resource requires reducing the other.

$$\alpha A / N^{\alpha} = \beta B / D^{\beta}$$

At the optimum: A small proportional change in model size has the same opposing effect as the forced change in data. This gives the balance equation above.

The allocation exponents: $N^* \propto C^a$ and $D^* \propto C^b$, where $a = \beta / (\alpha + \beta)$ and $b = \alpha / (\alpha + \beta)$. Thus $a + b = 1$.

When $\alpha \approx \beta$: The budget grows roughly equally in model size and data. The coefficients A and B still determine the token/parameter ratio.

Definitions on this slide

N*, D*: the preferred model size and token count at the chosen budget. * means optimal.

C = 6ND: the assumed dense-training compute constraint.

A, B, α, β: coefficients and decay exponents in the model and data loss terms.

a, b: budget-allocation exponents, distinct from the loss-decay exponents.

Marginal gain: the improvement obtained by a small additional change in a resource.

Intuition. If shifting a little budget from data to model size would lower loss, the current split is not optimal. The best split is where that local advantage disappears.

Derivation from $L(N,D) = E + A/N^{\alpha} + B/D^{\beta}$ under $C = 6ND$

One explanation: resolving a complex space

Some theories connect slow improvement to the number of directions in which the data varies.

$$L - E \propto N^{-\alpha}, \quad \text{with } \alpha = 4 / d$$

Manifold picture: The data may lie near a structured surface with d effective dimensions. More parameters represent progressively finer details on that surface.

Spacing: If N representational elements cover d dimensions, their typical separation scales as N raised to –1/d. High dimension makes that spacing shrink slowly.

Conditional theory: The exponent 4/d follows in particular smooth-target, resolution-limited settings. It is a possible mechanism, not a universal formula for language models.

Definitions on this slide

Manifold: a structured space that locally behaves like a lower-dimensional surface.

d: intrinsic dimension, the number of independent directions needed to describe that structure.

N / α: parameter count / positive loss-decay exponent.

L – E: prediction loss above the limiting floor E.

Resolution-limited: performance mainly limited by how finely a model represents the target.

Intuition. Twice as many map points help a lot on a line, less on a surface, and much less in a space with many independent directions.

Sharma & Kaplan (2020); Bahri et al. (2021)

Two explanations based on rare patterns and skills

Many small learning events can add up to a smooth aggregate curve.

Zipf-distributed features

Feature: a recurring pattern that helps prediction, such as a phrase or relation.

Zipf's law: frequency decreases roughly as a power of rank: the second-most-common feature is rarer than the first.

Learning mechanism: common patterns appear early. More data gradually reveals rare ones.

Quanta of skill

Quantum / skill: a discrete unit of useful behavior in the model's theoretical description.

Threshold: the training scale at which a particular skill becomes reliably usable.

Aggregate loss: the average prediction error across all skills and their frequencies of use.

What the theories imply

Power law: a regular relation between resource ratios and loss reductions.

Data distribution: the frequencies of patterns determine which learning opportunities remain.

Scope: architecture and training can also change slopes. Measure a new curve when the recipe changes.

Intuition. A city lights up one window at a time. Individual switches are abrupt, but the total light can rise smoothly when many switches turn on at different times.

Hutter (2021); Michaud et al. (2023); theoretical explanations, not a settled universal account

Serving cost can favor a smaller model trained longer

The best training-only choice can differ from the best choice over a model's lifetime.

$$C_{total} \approx 6ND + 2NQ$$

Training cost: 6ND: More training tokens D can improve a smaller model, while increasing its one-time training bill.

Serving cost: 2NQ: For a fixed number Q of generated tokens, a smaller parameter count N reduces approximate forward-pass compute every time the model is used.

Over-training: Training beyond the token/parameter ratio that minimizes training compute for a target loss. It can be economical when serving volume is large.

Definitions on this slide

N, D: model parameters and training tokens.

Q: total generated tokens expected over the model's deployed lifetime.

Inference / serving: using trained weights to generate predictions.

Objective: the quantity being minimized, here lifetime compute at a required quality.

Ctotal: training plus serving operations. It does not include all hardware or latency costs.

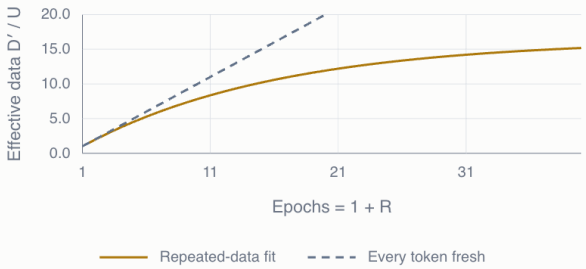
Intuition. Spending more once to teach a smaller model can pay back whenever it answers a question. The more often it is used, the more that recurring saving matters.

Sardana et al. (2024); Gadre et al. (2024)

Repeated data gives diminishing returns

A fitted effective-data model estimates how much fresh data repeated examples are worth.

$$D' = U\,[1 + R^*\,(1 - \exp(-R / R^*))]$$



With $R^* \approx 15.4$, this effective-data term approaches $16.4 \times U$ as repetition grows.

Definitions on this slide

U: unique tokens in the available dataset.

Epoch: one full pass over the dataset. R counts extra passes after the first.

D′: effective fresh-token count in the fitted loss model.

R*: the fitted repetition-decay scale, about 15.4 in this study.

exp(z): e raised to z, with $e \approx 2.718$.

Intuition. Rereading a book helps, especially at first. Later rereads add less new information. Repeated tokens still cost compute even when their learning value falls.

Muennighoff et al. (2023); fitted repetition curve

Hyperparameters also move with scale

DeepSeek LLM fit useful learning-rate and batch-size trends from smaller experiments.

$$\eta^* = 0.3118\ C^{-0.125} \qquad B^* = 0.2920\ C^{0.3271}$$

For 10× more compute: The fitted learning rate multiplies by about 0.75. The fitted batch multiplies by about 2.12.

Meaning of the trend: Longer or larger runs can benefit from smaller weight updates and more tokens per update, under the paper's training setup.

How to use it: Treat the fit as a starting point and test nearby settings. Its coefficients depend on units, model family and training recipe.

Definitions on this slide

Hyperparameter: a training setting chosen by the experimenter rather than learned as a weight.

η / learning rate: the multiplier controlling the size of an update.

B: batch size in tokens per update. * marks the fitted best value.

C: the paper's training-compute measure in FLOPs, with its stated accounting convention.

Sweep: several runs that systematically vary one or more settings.

Intuition. Large and small models need not take equally sized steps through the learning problem. A scaling experiment should compare well-tuned runners at every distance.

DeepSeek LLM (2024), §3.1; these are recipe-specific fits

Tuning and transfer keep a ladder comparable

A consistent experimental protocol can allow numerical settings to change predictably with model size.

Sweep and fit

Grid search: test a set of candidate settings, such as learning rate and batch size.

Response surface: the loss measured across those settings. A broad basin means many choices work similarly well.

Validation: check the proposed setting on a larger run before committing the full budget.

Transfer with μP

μP: maximal update parametrization: rules for scaling initialization and updates as width changes.

Initialization: the starting values of weights before learning.

Width transfer: under compatible assumptions, a learning rate tuned on a small model can work on a wider one.

Other settings

Weight decay: an update that tends to shrink weights, which can help control overfitting.

Dropout: randomly suppressing some internal values during training to discourage fragile reliance on them.

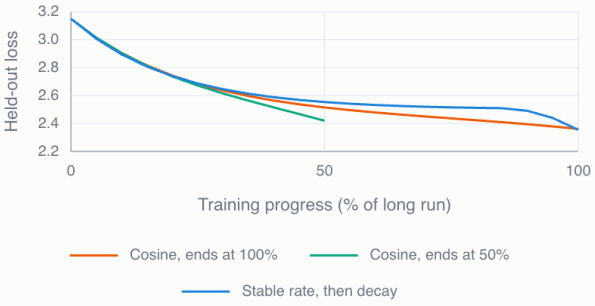
Regularization: methods that aim to improve performance on unseen data.

Intuition. Keeping the recipe consistent means keeping the rule for choosing settings consistent. It does not require giving every model identical numbers.

Yang et al. (2022); Porian et al. (2024); Microsoft AI (2026)

The learning-rate schedule changes the measured endpoint

A model halfway through a long run need not match a model trained to finish at that same token count.



A short decay branch can estimate an endpoint while reusing the shared earlier training.

Definitions on this slide

Schedule: the learning rate as a function of training progress.

Cosine decay: a smooth decrease shaped like part of a cosine wave.

Horizon: the planned number of tokens or updates before stopping.

WSD: warmup-stable-decay: raise the rate, hold it steady, then lower it.

Branch / checkpoint: continue from saved training state using a different ending schedule.

Intuition. A runner who knows the finish line is near changes pace. Compare runs at their intended finish, with comparable schedules, rather than treating every intermediate snapshot as equivalent.

Hu et al. (2024); Hägele et al. (2024); curves are illustrative

Sparse models separate stored size from work per token

A mixture-of-experts model activates only part of its capacity for each input.

$$N_{\text{total}} \neq N_{\text{active}}$$

Mixture of experts (MoE): Several specialized subnetworks share the work. A router chooses which experts process each token. Shared layers still run for all tokens.

Two different sizes: Total parameters influence storage and potential capacity. Active parameters more closely track the arithmetic per token.

Scaling question: At fixed compute, how many experts should there be and how large should each be? Studies fit laws with these choices as additional variables.

Definitions on this slide

Ntotal: all stored model parameters, including all experts.

Nactive: the parameters used to process one token, under the stated counting convention.

Expert / router: a subnetwork / the mechanism that selects subnetworks for each token.

Sparse: only a subset of available components activates for a given input.

Granularity: how finely capacity is divided into experts.

Intuition. A library can hold many books while you read only a few for one question. Storage capacity and work per question are different quantities.

Clark et al. (2022); Krajewski et al. (2024)

Vocabulary and precision change the resource tradeoff

A parameter count does not describe every important design choice.

Vocabulary

Vocabulary size: the number of distinct tokens the tokenizer can emit.

Embedding table: learned vectors representing those tokens. A larger vocabulary increases this table.

Tradeoff: larger tokens can shorten sequences, but a larger vocabulary adds parameters and output computation.

Numerical precision

Precision P: the number of bits used to represent a value in a specified format.

Quantization: rounding values to a smaller set of representable numbers.

N_eff = N f(P): a fitted analogy: lower precision can act like fewer effective parameters. f(P) describes the retained effectiveness.

Reading the evidence

N_eff / N: effective parameter count / actual parameter count.

Bit: a binary unit. More bits generally represent values more finely.

Conditional result: Kumar et al. estimated a training optimum near 7–8 bits in their setup. That is not a universal hardware recommendation.

Intuition. A bigger dictionary saves some spelling, but takes space. Coarser arithmetic saves resources, but can blur useful distinctions. Both choices can shift the best model/data balance.

Tao et al. (2024); Kumar et al. (2024)

Data mixing laws describe what the model practices

Changing the proportion of domains changes the learning problem, even at the same total token count.

$$r_1 + r_2 + \ldots + r_k = 1$$

A data mixture: Choose what share of training comes from each source, such as code, books or web text. A share of 0.20 means 20% of sampled tokens.

A domain-specific loss: Measure the model separately on held-out examples from each domain. Improving one domain can help, leave unchanged or hurt another.

A controlled study: Vary the shares across small runs, fit the relationship, then validate the proposed mixture at larger scale.

Definitions on this slide

Domain: a type or source of data with characteristic patterns.

r_i: the sampling fraction of domain i. i labels a domain.

k / Σ idea: k is the number of domains. Their fractions must add up to one.

Data quality: how useful and appropriate examples are for the target task, not simply their quantity.

Distribution shift: a change between the data used to fit a law and the data where it is applied.

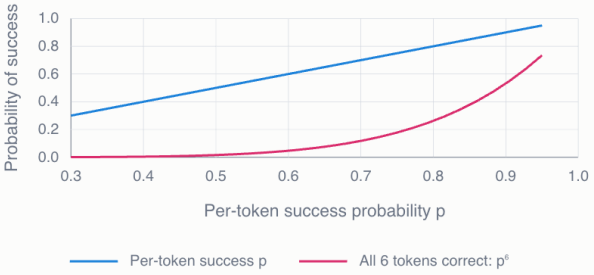
Intuition. Hours of practice do not tell you what was practiced. The same training budget can teach different strengths depending on which examples it sees.

Ye et al. (2024), Data Mixing Laws

Smooth improvement can look abrupt under a strict score

Toy example: six tokens each succeed independently with probability p. Exact match gives no partial credit.

Exact-match probability ≈ p^6



At p = 0.5, all-six success is 1.6%. At p = 0.9, it is 53.1%. The curve is still continuous.

Definitions on this slide

p: probability of success for one token in this simplified example.

Exact match: a score of one only if the entire answer matches, and zero otherwise.

Benchmark: a fixed task set and scoring procedure used to compare models.

Emergence: an ability that appears to arise abruptly as scale grows.

Continuous metric: a score that can record gradual changes, such as average prediction loss.

Intuition. A six-question quiz scored “all correct or zero” hides partial learning. A student can improve steadily while their perfect-score rate stays low for a long time.

Wei et al. (2022); Schaeffer et al. (2023); toy example with explicit independence assumptions

A two-stage map connects compute to task accuracy

A smooth intermediate quantity can be easier to forecast than a score bounded between zero and one.

1. Compute to task loss

Compute C: the total operations used to train the model.

Task NLL: negative log-likelihood of the correct answers on a benchmark. Smaller means the model assigns them more probability.

Normalized NLL: divide by a consistent unit, such as characters, so answer lengths are comparable.

2. Task loss to score

Accuracy: the fraction of benchmark answers scored correct.

Sigmoid: an S-shaped function that can map a continuous input to a bounded score.

Calibration: fit this mapping using models with both measured task loss and measured accuracy.

3. Combine the maps

Forecast: predict task loss for the target compute, then convert that loss to a predicted score.

Extra evidence: older large models may help calibrate the second relationship.

Scope: the mapping can change with benchmark, prompting, data mix or training stage.

Intuition. A thermometer gives a continuous reading before a yes/no alarm sounds. Forecasting the reading first can make the alarm outcome easier to anticipate.

Llama 3 report (2024), §3.2.1; Bhagia et al. (2024)

Large forecasts have worked in specific studies

Successful extrapolations are evidence for a method within a recipe, not a universal safe distance.

Study	Small-run compute relative to target	Predicted outcome
GPT-4	At most 1 / 10,000	Held-out final loss
GPT-4	At most 1 / 1,000	HumanEval pass rate
Llama 3	Top rung: 10^{22} FLOPs Target: 3.8×10^{25}	Task accuracy via a two-stage map

Definitions on this slide

Held-out: reserved for evaluation rather than training.

HumanEval: programming problems scored by whether generated code passes tests.

Pass rate: fraction of tested problems or samples that satisfy the scoring rule.

Compute ratio: $3.8 \times 10^{25} / 10^{22} = 3{,}800\times$, about 3.6 decades.

Llama 3's scaling experiments ranged from 6×10^{18} to 10^{22} FLOPs. B in "405B" means billion parameters.

Intuition. A carefully calibrated ruler can measure beyond its first markings. Its past success does not guarantee that the object ahead will keep the same shape.

GPT-4 Technical Report (2023), §3; Llama 3 report (2024), §3.2.1

A scaling ladder is a controlled family of training runs

A rung is one measured model at one scale. The ladder connects several rungs with a fitted relationship.

For a fixed ratio r: $D = rN$, $C \approx 6rN^2$

Keep the family consistent: Use the same data sources and proportions, tokenizer, architecture rules, optimizer family and evaluation.

Scale according to a stated rule: Increase model size N and training tokens D together. Tune learning rate and batch using a consistent protocol.

Match the question: One token/parameter ratio can forecast that recipe. Several token budgets per size are needed to study the best allocation.

Definitions on this slide

N, D, C: parameters, training tokens and training operations.

r = D / N: tokens per parameter, sometimes abbreviated TPP.

Recipe: the complete set of data, architecture and training choices.

Rung: one run at a specified scale, tuned and evaluated under the common protocol.

Target: the larger run whose result the ladder aims to predict.

Intuition. A ladder lets you test whether a trend persists as the experiment gets harder. A collection of unrelated models does not isolate the effect of scale.

Model-ladder methodology across Llama 3, OLMo studies and Microsoft AI

A ladder needs enough span and an honest test

These are design heuristics. The required evidence depends on the size of the decision.

Choice	Starting point	Reason
Coverage	5–8 rungs over a broad compute range	Several points must constrain the curve
Spacing	Geometric spacing, such as 3–4× compute	Equal ratios sample log space evenly
Holdout	Reserve the largest measured rung	Test prediction before fitting it
Replication	Repeat small rungs with multiple seeds	Estimate variation from random training
Shape rule	Tie width and heads to layer count	Keep a comparable architecture family

Definitions on this slide

Geometric spacing: multiply by the same factor between neighboring rungs.

Holdout rung: a completed run excluded from the fit and used to test its prediction.

Seed: the random initialization or sampling setting for a run.

Layers / width / heads: stacked blocks / representation size / parallel attention components.

Five rungs with 4× spacing span $4^4 = 256\times$ compute. Five with 2× spacing span only 16×.

Intuition. Two nearby points can suggest a line in almost any direction. Broad coverage and an unseen test reveal whether the apparent slope is trustworthy.

Experimental-design guidance; adapt to the recipe and decision

Fitting the curve means testing several plausible explanations

A good-looking line is the beginning of a forecast, not its validation.

$$L(C) = E + A\,C^{-\alpha}$$

Fit the final measured loss: Choose positive A and α, and a sensible floor E. Compare predicted and measured log loss, then search for the best constants.

Use robust optimization: A Huber penalty reduces the influence of unusually bad points. L-BFGS searches for parameters using local slope information. Try several starting values.

Inspect what the fit misses: Plot residuals, refit with each rung omitted, and predict the held-out top rung. Curvature can signal a wrong form or an inconsistent run.

Definitions on this slide

L, C: held-out loss and training operations.

E, A, α: floor, curve scale and positive decay exponent.

Residual: measured value minus fitted value, in the units used for fitting.

Huber loss: a fitting penalty that is quadratic near zero and grows linearly for large errors.

L-BFGS: a memory-efficient numerical optimization method for smooth objectives.

Intuition. Several curves can pass through the same small-scale points. A forecast becomes more credible when those curves agree on an unseen larger run.

Hoffmann et al. (2022), fitting procedure; Besiroglu et al. (2024), fit uncertainty

Small loss errors can mean large compute errors

Compute equivalence depends on the reducible part of loss and the fitted slope.

$$m = (q_1 / q_2)^r, \quad \text{with } r = 1 / \alpha$$

Compute multiplier m: The operations needed to reduce excess loss from q_1 to q_2 along the same fitted curve. Here q = L – E.

Example: If α = 0.08, a 1% ratio change in excess loss gives $1.01^{12.5} \approx 1.13\times$ compute. A 5% change gives about 1.84×.

Uncertainty: Vary the floor and curve form, repeat fits and test held-out rungs. A spread of refits is a sensitivity band, not automatically a calibrated confidence interval.

Definitions on this slide

q_1, q_2: starting and target excess losses, both strictly above zero.

L / E: measured loss / fitted floor.

α: the positive decay exponent for excess loss versus compute.

m: ratio of target compute to starting compute.

r: inverse exponent, r = 1 / α. A smaller α gives a larger r.

Intuition. On a shallow slope, a small vertical shift corresponds to a long horizontal move. That is why a seemingly tiny loss advantage can imply a large and uncertain compute saving.

Algebra from $q = A\,C^{-\alpha}$; numerical examples are derived

A fictional $1M study: dollars to operations

The price and utilization below are planning assumptions, not a current hardware quote.

$$\text{Useful FLOPs} = \text{hours} \times 3{,}600 \times \text{peak rate} \times \text{MFU}$$

Rental budget: At an assumed $2.50 per GPU-hour, $1,000,000 buys 400,000 GPU-hours.

Useful work per hour: 989×10^{12} FLOPs/s $\times$ 0.40 $\times$ 3,600 $\approx$ 1.424×10^{18} FLOPs per GPU-hour.

Total compute: $400{,}000 \times 1.424 \times 10^{18} \approx 5.70 \times 10^{23}$ useful training FLOPs. Real small runs may achieve much less utilization.

Definitions on this slide

GPU-hour: one graphics processing unit running for one hour.

H100: the NVIDIA accelerator used in this fictional budget.

TFLOPS: 10^{12} floating-point operations per second, a rate rather than a total.

BF16: bfloat16, a 16-bit numerical format. The assumed peak rate is 989 TFLOPS.

MFU: model FLOPs utilization: useful model operations divided by theoretical hardware capacity.

Intuition. A vehicle's top speed does not tell you its average speed on a trip. Peak arithmetic throughput needs a realistic utilization factor before it becomes a budget.

Fictional budget using the supplied deck's assumptions; calculations shown explicitly

The study's rungs and their estimated cost

All rungs use the illustrative ratio D = 20N. Costs assume the same throughput at every scale.

Rung	N	D	FLOPs	GPU-hours
1	50M	1B	3.0×10^{17}	0.21
2	150M	3B	2.7×10^{18}	1.90
3	400M	8B	1.92×10^{19}	13.48
4	1B	20B	1.20×10^{20}	84.26
5	3B	60B	1.08×10^{21}	758.34
Holdout	8B	160B	7.68×10^{21}	5,392.65

Definitions on this slide

N / D: parameter count / training tokens.

M / B: million (10^6) / billion (10^9).

FLOPs: total floating-point operations, estimated as 6ND.

GPU-hours: operations divided by the assumed 1.424×10^{18} operations per GPU-hour.

Rungs 1–5 total about 858 GPU-hours for one recipe and one seed. Displayed costs are rounded.

Intuition. The largest rung costs more than all the smaller rungs combined. Small experiments are cheap, but the top rung supplies crucial evidence about scale.

Derived estimates under the fictional budget assumptions

What the $1M ladder can establish

A small fraction of the budget can test a forecast, but each additional scientific question costs more.

Three recipes, two seeds: $6 \times 858 \approx 5{,}150$ GPU-hours for rungs 1–5. One 8B holdout adds about 5,393 hours. Together: about 2.64% of the 400,000-hour budget.

A possible target: A 30B model trained on 600B tokens costs 1.08×10^{23} FLOPs, about 75,834 GPU-hours or 18.96% of this budget.

What remains unproven: A fixed D/N ladder cannot locate the best token/parameter ratio. That needs several token budgets at each model size. A single holdout tests only its own recipe.

Definitions on this slide

Recipe variant: one controlled change, such as a different data mix.

Seed: a choice of random starting weights or example order.

Holdout: a larger run reserved to test the fitted prediction.

N / D / B: parameters / training tokens / billion (10^9).

Extrapolation distance: the target is 100× above rung 5, about 14× above the 8B holdout.

Intuition. A pilot study is valuable when it answers the decision you face. A cheap curve at one token ratio cannot answer every question about the best training plan.

Derived study costs; fixed-ratio ladder and holdout limitations

Pitfalls in the training experiment

An inconsistent run can bend a fitted curve for reasons unrelated to scale.

Unmatched endpoints

Schedule: the rule changing the learning rate over training.

Failure: an intermediate point in a long run may not match a run scheduled to finish there.

Repair: use comparable endpoints or validate short warmup-stable-decay branches.

Untuned rungs

Hyperparameters: settings such as learning rate and batch size, chosen before training.

Failure: copying a small model's settings can disadvantage a larger model, or vice versa.

Repair: apply a scale-aware tuning protocol and inspect unusually high-loss rungs.

Confounded comparisons

Confounder: another changed factor that could explain the result.

Failure: changing data, precision and architecture together makes attribution unclear.

Repair: change one factor per comparison and count compute consistently.

Intuition. To learn whether a bigger engine helps, keep the road and driving test comparable. Otherwise the experiment measures several changes at once.

Experimental pitfalls discussed in the supplied deck; Porian et al. (2024)

Pitfalls in evaluation and extrapolation

A clean training experiment can still produce a misleading forecast if the measurement changes.

Contaminated evaluation

Contamination: evaluation examples or close copies appear in training.

Failure: the model can appear to generalize when it is recalling familiar material.

Decontamination: detect and remove training/evaluation overlap before comparing runs.

Incompatible units

Tokenizer: the procedure that divides text into model tokens.

Failure: nats per token can change when tokens have different lengths, even on the same text.

Common metric: bits per byte divides total loss in nats by text bytes and by $\ln(2) \approx 0.693$, the natural log of 2.

Overconfident forecasts

Floor E: the fitted limiting loss, often weakly constrained by small runs.

Failure: small-scale rankings can reverse, and long extrapolations can depend strongly on E.

Repair: compare at the target scale, reserve a larger test run and report sensitivity.

Intuition. A better score is meaningful only when the exam is equally unfamiliar and the scoring ruler has the same units.

Evaluation and forecast checks drawn from the supplied lecture

What remains outside a pretraining scaling law

A law predicts the quantity it was fitted to, under the conditions that generated its evidence.

Changing the data

Curation: selecting, filtering and balancing training examples.

Synthetic data: examples produced by a model or another generation procedure.

Constraint: repetition and quality determine useful information, so raw token count can be insufficient.

Changing the model

Sparse model: a model using only some of its capacity for each input.

Active / total size: weights used per token / all stored weights. Both can matter.

Recipe law: a relationship that also models choices such as vocabulary, precision or tuning.

Changing the task stage

Post-training: additional training after broad pretraining, often for instructions or preferences.

Inference-time compute: work spent generating, checking or searching over answers after training.

Limitation: these can change capability in ways a pretraining-loss fit does not predict.

Intuition. A fuel-consumption curve for one driving condition cannot by itself predict every route. Define the training stage, target metric and conditions before using the law.

MAI-Base-2: a ladder that also scales data repetition

Microsoft's October 2026 report describes holding tokens per active parameter and data reuse comparable across scales.

The model: Microsoft reports 2.8T total parameters, 80B active parameters and 50T training tokens. The run used 16,000 GB200 GPUs for 38 days.

The size rule: A layer count ℓ determines width approximately as $(256/3) \times \ell$. The query-head count follows ℓ, rounded to a multiple of 16.

Source scaling: If the full run repeats a source eight times, a run with one-tenth as many tokens repeats one-tenth as much unique source data eight times too.

Definitions on this slide

B / T: billion (10^9) / trillion (10^{12}).

Active / total parameters: weights used per token / all stored weights in the mixture-of-experts model.

ℓ / width / query heads: layer count / internal vector size / parallel attention components.

GB200 GPU: a type of NVIDIA accelerator used for training.

Source scaling: restricting unique data at small scales to match the full run's repetition pattern.

Intuition. A small rehearsal should practice the same amount of repetition as the full performance. Otherwise it can reward choices that fail once the large run starts rereading data.

Microsoft AI, "Frontier Pre-training at Microsoft AI," October 15, 2026

MAI-Base-2: forecasting and comparing efficiency

The forward forecast and the retrospective efficiency comparison answer different questions.

Forecast result: Microsoft reports that its smaller-model scaling ladder accurately predicted the larger model's held-out evaluation.

Efficiency comparison: The reported 4.2–12.3× FLOPs-efficiency range compares compute needed to match DeepSeek-V4-Pro-Base on four held-out evaluations.

Important distinction: Those comparison fits include the final MAI-Base-2 result. The matching points lie inside the fitted range. The efficiency comparison is not itself an unseen final-model forecast.

Definitions on this slide

Held-out: data reserved for evaluation rather than training.

BPB: bits per byte, a prediction-loss measure normalized by text bytes.

FLOPs-efficiency: relative operation counts needed to reach the same evaluation loss.

Interpolation: estimating inside the measured range.

Extrapolation: predicting beyond the measured range.

Intuition. Predicting the finishing time before a race and comparing runners after the race are both useful. They supply different kinds of evidence.

Microsoft AI (2026), "Robust scaling ladders" and "Isolating modeling efficiency from compute"

Exercises: allocation, compute equivalence and repetition

Use the stated assumptions. Each question isolates one idea.

1. Allocate a budget

Given: training compute $C = 10^{22}$ FLOPs. Use $C = 6ND$ and $D = 20N$.

Find: model parameters N and training tokens D.

Then consider serving: $Q = 10^{14}$ generated tokens adds $2NQ$ operations. What extra information is needed to choose the best lifetime-cost model?

2. Convert a loss gap

Given: a fitted law $L = E + A\,C^{-\alpha}$ with $\alpha = 0.08$.

Excess loss: $q = L - E$ falls from 0.92 to 0.90. E is the floor and A is the curve's scale.

Find: the compute multiplier: raise $0.92 / 0.90$ to the power $1 / \alpha$. Explain why the floor matters.

3. Value repeated data

Given: $U = 5T$ unique tokens and eight passes. T means trillion. Extra passes $R = 7$.

Fit: $D' = U[1 + R^*(1 - \exp(-R/R^*))]$, with repetition scale $R^* = 15.4$.

Find: effective fresh tokens D' and their fraction of 40T. $\exp(z)$ means e raised to z.

Intuition. The same word "scale" hides different operations: splitting a budget, inverting a curve and discounting repeated information.

Exercises adapted from the supplied lecture

Exercises: building and deliberately breaking a ladder

These exercises require actual training runs. They are experiments to perform, not results claimed here.

4. Build: Train five model sizes from roughly 1M to 50M parameters on one corpus, with 20 tokens per parameter. Use comparable warmup-stable-decay endpoints.

Fit and test: Fit $L = E + A\,C^{-\alpha}$ on the lower four rungs, predict the fifth, and compare with its measured loss. Repeat fits leaving out each lower rung.

5. Break: For each model size, also train a longer cosine-schedule run and read its intermediate loss at the intended budget. Refit using these unmatched endpoints and compare forecast errors.

Definitions on this slide

M / corpus: million (10^6) / the collection of training text.

WSD: warmup-stable-decay learning-rate schedule.

L, C, E, A, α: loss, compute, floor, scale coefficient and positive decay exponent.

Leave-one-out refit: repeat the fit while omitting one measured rung at a time.

Holdout error: the difference between predicted and observed loss on the excluded top rung.

Intuition. An intentionally flawed experiment makes the control variables visible. If the fitted slope changes, ask whether the model family changed or only the measurement protocol did.

Experimental exercises; no training runs were performed for this deck

Worked answer 1: training allocation and serving cost

The training-only estimate is determined. A lifetime-cost optimum needs a quality target and a fitted loss model.

$$N = \sqrt{(10^{22} / 120)} \approx 9.13 \times 10^{9}$$

Training-only allocation: $D = 20N \approx 1.83 \times 10^{11}$ tokens, or about 183B. This uses the illustrative twenty-token rule.

Serving that model: With $Q = 10^{14}$ generated tokens, $2NQ \approx 1.83 \times 10^{24}$ FLOPs, far larger than the 10^{22} training budget.

Why "choose 3B" is not yet an answer: A 3B model would save about 1.23×10^{24} serving FLOPs, but we must know whether it can reach the required loss and how much training it needs.

Definitions on this slide

N, D: model parameters and training tokens.

C / Q: training operations / lifetime generated tokens.

B: billion (10^{9}).

Lifetime cost: approximately 6ND + 2NQ, subject to a required quality.

If 10^{22} is the total budget: serving alone requires $N < 50M$, leaving still less room for training.

Intuition. A cheaper model per answer can justify more training, but cheap answers are useful only if the model still meets the quality requirement.

Derived arithmetic; correction of the underdetermined serving extension in the source

Worked answer 2: a 0.02-nat gap can mean 1.32× compute

The calculation uses the excess loss above the fitted floor.

$$m = (0.92 / 0.90)^{12.5} \approx 1.316$$

Exponent: α = 0.08, so 1 / α = 12.5. The shallow slope amplifies the loss ratio.

Interpretation: Along this same law, reducing excess loss from 0.92 to 0.90 requires about 31.6% more compute. A recipe saving that work uses about 24% less compute.

Why the floor matters: If total losses were 2.62 and 2.60, using them directly would give only about 1.10×. That treats the irreducible floor as if compute could remove it.

Definitions on this slide

Nat: the loss unit from the natural logarithm of probability.

E: the fitted limiting loss.

q = L – E: excess loss, the reducible term in the fitted equation.

α: the decay exponent in $q = A\,C^{-\alpha}$. A sets the scale and C is compute.

m: the ratio of the two compute requirements.

Intuition. The useful comparison is how much unfinished work remains. Including the already unavoidable part makes the relative improvement look too small.

Derived from the exercise's illustrative α = 0.08 and excess losses 0.92, 0.90

Worked answer 3: eight passes give about 33T effective tokens

The model discounts repetition smoothly rather than treating every reread as fully new information.

$$D' = 5T \times [1 + 15.4(1 - \exp(-7/15.4))]$$

Count extra passes: Eight total passes means one first pass plus R = 7 repeats. The total processed data is 8 × 5T = 40T tokens.

Apply the fitted discount: exp(−7/15.4) ≈ 0.635. The effective multiplier is about 6.625, so D′ ≈ 33.13T.

Compare with fresh data: 33.13 / 40 ≈ 0.828, or 82.8%. Under this fitted term, eight passes provide about 17.2% less effective data than 40T fresh tokens.

Definitions on this slide

U = 5T: unique tokens, with T = trillion (10^{12}).

R = 7: passes beyond the first.

R* = 15.4: the paper's fitted repetition-decay scale.

D′: fresh-token equivalent in the fitted model, not a count of unique facts learned.

exp(z): the exponential function e^z, where e ≈ 2.718.

Intuition. The second reading may reveal things you missed. The eighth still helps, but it cannot contain as much new information as another book.

Derived using Muennighoff et al. (2023), repetition term

What to remember

A scaling law is a conditional empirical forecast, supported by a carefully controlled experiment.

Resources and limits

N, D, C: model parameters, training tokens and training operations.

Loss and floor: L measures prediction error. E is the fitted limit. Excess loss L – E often follows a power law.

Allocation: the best split depends on data and serving demand. Twenty tokens per parameter is a starting estimate.

The experiment

Ladder: comparable runs at several scales, each tuned under a common protocol.

Holdout: a larger run reserved to test a forecast before adding it to the fit.

Consistency: keep data, compute counting and evaluation units comparable. Match repetition and schedule behavior.

The interpretation

Capability: task performance, which may map nonlinearly from prediction loss.

Uncertainty: include sensitivity to the floor, slope and fitted form.

Compute equivalence: a small excess-loss advantage can imply a large compute saving, with an equally important error range.

Intuition. Small experiments are useful when they preserve the structure of the large decision. Definitions make the curve interpretable; controls make it credible.

Reading sequence: Kaplan (2020), Hoffmann (2022), Porian (2024), Muennighoff (2023), Llama 3 (2024)

Sources: history, core laws and theory

arXiv is a public research archive. “Et al.” means “and coauthors.” NIPS and ACL name research conferences.

Cortes et al. (1993)
Learning Curves: Asymptotic Values and Rate of Convergence. NIPS 6.

Banko & Brill (2001)
Scaling to Very Very Large Corpora for Natural Language Disambiguation. ACL.

Halevy, Norvig & Pereira (2009)
The Unreasonable Effectiveness of Data. IEEE Intelligent Systems 24(2).

Sun et al. (2017)
Revisiting Unreasonable Effectiveness of Data in Deep Learning Era. 1707.02968

Hestness et al. (2017)
Deep Learning Scaling is Predictable, Empirically. 1712.00409

Rosenfeld et al. (2019)
A Constructive Prediction of Generalization Error Across Scales. 1909.12673

Amodei & Hernandez (2018)
AI and Compute. Historical analysis of the largest training runs.

Kaplan et al. (2020)
Scaling Laws for Neural Language Models. 2001.08361

Hoffmann et al. (2022)
Training Compute-Optimal Large Language Models. 2203.15556

Porian et al. (2024)
Resolving Discrepancies in Compute-Optimal Scaling. 2406.19146

Besiroglu et al. (2024)
Chinchilla Scaling: A replication attempt. 2404.10102

Bahri et al. (2021)
Explaining Neural Scaling Laws. 2102.06701

Hutter (2021); Michaud et al. (2023)
Learning Curve Theory (2102.04074); Quantization Model (2303.13506).

Intuition. An empirical fit summarizes measurements. A theory proposes why a pattern occurs. A replication tests whether the original result survives a new analysis.

Sources: refinements and training design

A recipe-specific result applies to the study's data and training setup. "Et al." means "and coauthors."

McCandlish et al. (2018)
An Empirical Model of Large-Batch Training. 1812.06162

Sardana et al. (2024; first submitted 2023)
Beyond Chinchilla-Optimal: Accounting for Inference. 2401.00448

Gadre et al. (2024)
Language models scale reliably with over-training and on downstream tasks. 2403.08540

Muennighoff et al. (2023)
Scaling Data-Constrained Language Models. 2305.16264

Villalobos et al. (2022, later revisions)
Will we run out of data? 2211.04325

DeepSeek-AI (2024)
DeepSeek LLM. 2401.02954

Yang et al. (2022)
Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer. 2203.03466

Hu et al. (2024)
MiniCPM. 2404.06395

Hägele et al. (2024)
Scaling Laws Beyond Fixed Training Durations. 2405.18392

Clark et al. (2022)
Unified Scaling Laws for Routed Language Models. 2202.01169

Krajewski et al. (2024)
Scaling Laws for Fine-Grained Mixture of Experts. 2402.07871

Ye et al. (2024)
Data Mixing Laws: Optimizing Data Mixtures by Predicting Language Modeling Performance. 2403.16952

Intuition. Training compute, serving cost and limited unique data are different constraints. Choose the paper whose experiment matches the constraint you actually face.

Sources: design choices, capability and the case study

§ means a numbered section. “Illustrative” curves explain a relationship rather than show experimental measurements.

Tao et al. (2024)
Scaling Laws with Vocabulary: Larger Models Deserve Larger Vocabularies. 2407.13623

Kumar et al. (2024)
Scaling Laws for Precision. 2411.04330

Wei et al. (2022)
Emergent Abilities of Large Language Models. 2206.07682

Schaeffer, Miranda & Koyejo (2023)
Are Emergent Abilities of Large Language Models a Mirage? 2304.15004

OpenAI (2023)
GPT-4 Technical Report, §3. 2303.08774

Llama Team (2024)
The Llama 3 Herd of Models, §3.2.1. 2407.21783

Bhagia et al. (2024)
Task Scaling Laws via Compute-Efficient Model Ladders. 2412.04403

Microsoft AI (October 15, 2026)
Frontier Pre-training at Microsoft AI. Official report.

About the calculations
The $1M study is fictional. Costs and exercise answers are derived from the stated assumptions.

About the figures
Analytic curves reproduce equations or show labeled toy relationships. They do not claim newly measured model results.

Intuition. A forecast, an observed outcome and an illustration are different kinds of evidence. Keeping those labels visible prevents a useful picture from claiming more than it shows.